

О проекте

Анализ поведения пользователей в интернет-магазине электроники.

Цель: понять, на каком этапе воронки магазин теряет пользователей, оценить удержание по когортам.

Задачи

- Загрузить и подготовить данные о событиях пользователей;
 - Построить воронку от просмотра до покупки;
 - Посчитать конверсию между этапами и найти самую слабую точку;
 - Сделать когортный анализ. Разбить пользователей по месяцам и посчитать Retention;
 - Посмотреть, как ведут себя пользователи в разрезе сессий и категорий товаров;
 - Сформулировать выводы и рекомендации.
-

Данные

Взял датасет с Kaggle — события пользователей в интернет-магазине электроники.

events.csv

- `event_time` — время события;
 - `event_type` — тип события (`view` , `cart` , `purchase`);
 - `product_id` — ID товара;
 - `category_id` — ID категории;
 - `category_code` — название категории;
 - `brand` — бренд;
 - `price` — цена;
 - `user_id` — ID пользователя;
 - `user_session` — ID сессии.
-

Этапы работы

1. Загрузка и просмотр данных
 2. Анализ воронки
 3. Когортный анализ и Retention
 4. Выводы и рекомендации
-

Инструменты

Python

- `Pandas`
- `Matplotlib`
- `seaborn`

Источник данных: [Kaggle](#)

Загрузка и просмотр данных

```
In [1]: import pandas as pd
pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', '{:,.2f}'.format)

import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: events = pd.read_csv('data/events.csv')
start_size = events.shape[0]
```

```
In [3]: events.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 885129 entries, 0 to 885128
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   event_time             885129 non-null object  
1   event_type             885129 non-null object  
2   product_id             885129 non-null int64   
3   category_id            885129 non-null int64   
4   category_code          648910 non-null object  
5   brand                  672765 non-null object  
6   price                  885129 non-null float64  
7   user_id                885129 non-null int64   
8   user_session           884964 non-null object  
dtypes: float64(1), int64(3), object(5)
memory usage: 60.8+ MB
```

```
In [4]: events.sample(5)
```

```
Out[4]:
```

	event_time	event_type	product_id	category_id	category_code	bra
130164	2020-10-21 19:15:08 UTC	view	3758903	2144415923107266682	computers.peripherals.printer	
76104	2020-10-11 21:33:35 UTC	view	749087	2144415935732122060	NaN	N
879851	2021-02-27 23:52:39 UTC	view	1282166	2144415973346640379	computers.components.hdd	samsu
352496	2020-11-25 19:38:53 UTC	view	459699	2144415927158964449	NaN	N
696047	2021-01-28 08:41:11 UTC	view	264744	2144415922939494519	NaN	panaso

```
In [5]: events.isna().sum()
```

```
Out[5]: event_time      0
        event_type      0
        product_id      0
        category_id     0
        category_code    236219
        brand            212364
        price            0
        user_id          0
        user_session     165
        dtype: int64
```

```
In [6]: events['event_time'].min()
```

```
Out[6]: '2020-09-24 11:57:06 UTC'
```

```
In [7]: events['event_time'].max()
```

```
Out[7]: '2021-02-28 23:59:09 UTC'
```

Вывод

В таблице 885159 строк. Имеются пропуски в `category_code`, `brand` и `user_session`.

Строки с пропусками в `user_session` не подойдут для дальнейшего анализа, от них нужно будет избавиться.

Предобработка

Приведем время в формате `datetime` и добавим дополнительные столбцы.

```
In [8]: events['event_time'] = pd.to_datetime(events['event_time'])

events['event_date'] = events['event_time'].dt.date
events['event_month'] = events['event_time'].dt.to_period('M')
events['event_week'] = events['event_time'].dt.to_period('W')
```

Избавимся от строк без `id` сессии и дубликатов

```
In [9]: events.dropna(subset=['user_session'], inplace=True)
```

```
In [10]: print('Кол-во дублей:', events.duplicated().sum())
events = events.drop_duplicates()
```

Кол-во дублей: 652

Приведем `id` в формате `object`

```
In [11]: events[['product_id', 'category_id', 'user_id']] = events[['product_id', 'category_id', 'user_
```

Статистики по столбцам

```
In [12]: events.describe(include='object').T
```

Out[12]:

	count	unique	top	freq
event_type	884312	3	view	792943
product_id	884312	53452	1821813	14551
category_id	884312	718	2144415922427789416	116606
category_code	648311	107	computers.components.videocards	116606
brand	672118	999	asus	27657
user_id	884312	407237	1515915625554995474	572
user_session	884312	490398	nFlhu5QzOd	572
event_date	884312	158	2020-11-18	8023

```
In [13]: print('Удалено строк', start_size - events.shape[0])
print('Доля оставшихся строк:', round(events.shape[0] / start_size, 4))
```

Удалено строк 817

Доля оставшихся строк: 0.9991

Анализ пользовательской активности

```
In [14]: daily_events = events.groupby('event_date')['event_type'].count()

daily_events.plot(kind='line', figsize=(10,5))

plt.title('Количество событий по датам')
plt.xlabel('Дата')
plt.ylabel('Количество событий в день')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



```
In [15]: daily_events.describe()
```

```
Out[15]: count    158.00
mean    5,596.91
std     990.36
min     2,262.00
25%     4,928.00
50%     5,650.50
75%     6,300.50
max     8,023.00
Name: event_type, dtype: float64
```

Ежедневная пользовательская активность составляет около 5600 событий. Минимальное количество событий приходится на новогодние праздники - 2262. Примерно в середине-конце ноября активность уменьшается, а в начале января возрастает обратно

Продуктовая воронка

```
In [16]: funnel = events.groupby('event_type')['user_id'].nunique().reindex(['view', 'cart', 'purchase'])
```

```
In [17]: funnel_conv = pd.DataFrame({
    'users': funnel,
    'conversion': [
        1,
        funnel['cart'] / funnel['view'],
        funnel['purchase'] / funnel['cart']
    ]
})
funnel_conv
```

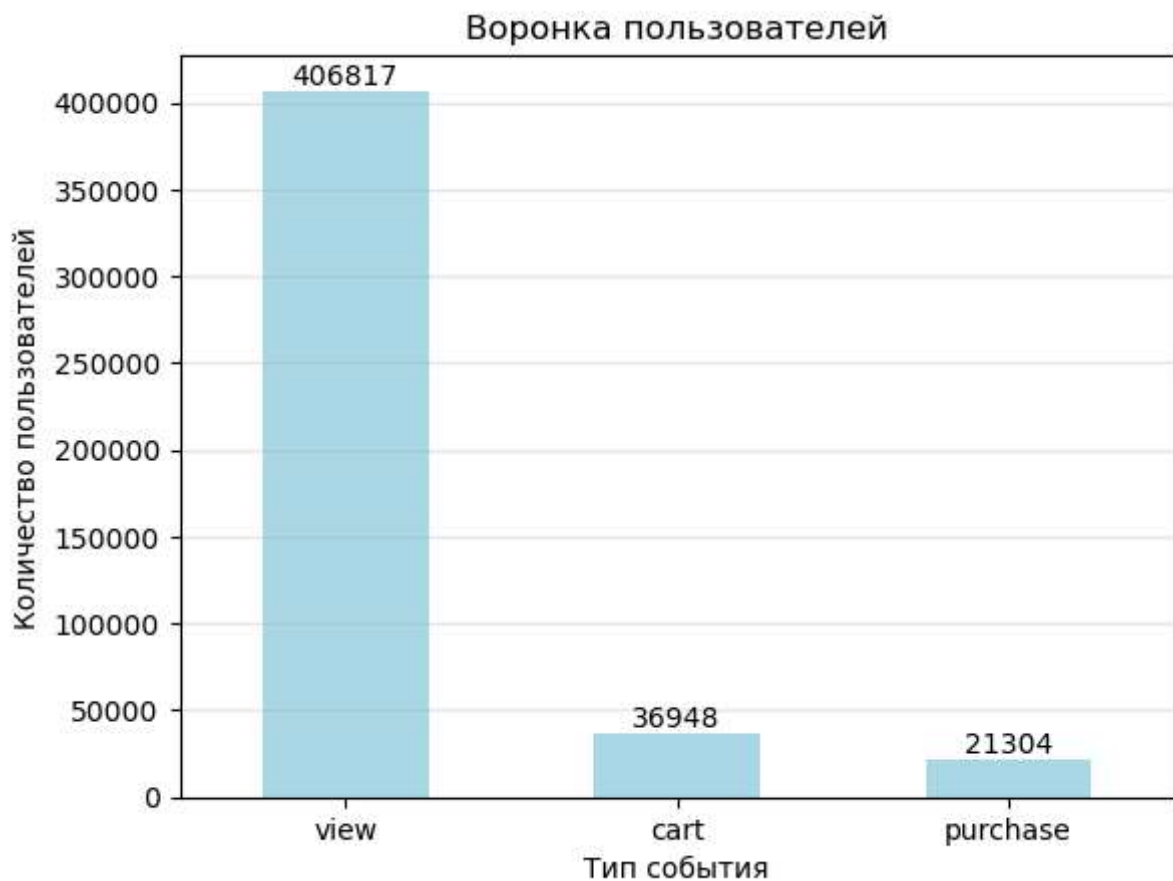
```
Out[17]:
```

	users	conversion
event_type		
view	406817	1.00
cart	36948	0.09
purchase	21304	0.58

```
In [18]: funnel_conv.plot(kind='bar', y='users', rot=0, color='lightblue', legend=False)

for i, v in enumerate(funnel_conv['users'].values):
    plt.text(i, v, str(v), ha='center', va='bottom')

plt.title('Воронка пользователей')
plt.xlabel('Тип события')
plt.ylabel('Количество пользователей')
plt.grid(axis='y', alpha=0.3)
plt.show()
```



Вывод

Основная проблема воронки находится на этапе перехода от просмотра товара к добавлению в корзину. Только **9.08%** пользователей после просмотра добавляют товар в корзину. При этом среди пользователей, добавивших товар в корзину, конверсия в покупку составляет **57.66%**.

Когортный анализ событий пользователей

Формируем когорты по месяцу первого события

```
In [19]: first_event = events.groupby('user_id')['event_time'].min().reset_index()
first_event['cohort_month'] = first_event['event_time'].dt.to_period('M')

events = events.merge(first_event[['user_id', 'cohort_month']], on='user_id', how='left')
```

Размер когорт

```
In [20]: cohort_size = events.groupby('cohort_month')['user_id'].nunique()
cohort_size
```

```
Out[20]: cohort_month
2020-09    15333
2020-10    83249
2020-11    90089
2020-12    69467
2021-01    78270
2021-02    70829
Freq: M, Name: user_id, dtype: int64
```

Определяем время жизни пользователя - номер месяца относительно когорты

```
In [21]: events['lifetime'] = (events['event_month'].astype(int) - events['cohort_month'].astype(int))
```

Количество активных пользователей по когортам

```
In [22]: cohort_users = events.groupby(['cohort_month', 'lifetime'])['user_id'].nunique().reset_index()
```

```
In [41]: cohort_users['retention'] = cohort_users['user_id'] / cohort_users['cohort_month'].map(cohort_users[['cohort_month', 'lifetime']].drop_duplicates())
```

```
Out[41]:
```

	cohort_month	lifetime
0	2020-09	0
1	2020-09	1
2	2020-09	2
3	2020-09	3
4	2020-09	4
5	2020-09	5
6	2020-10	0
7	2020-10	1
8	2020-10	2
9	2020-10	3
10	2020-10	4
11	2020-11	0
12	2020-11	1
13	2020-11	2
14	2020-11	3
15	2020-12	0
16	2020-12	1
17	2020-12	2
18	2021-01	0
19	2021-01	1
20	2021-02	0

Матрица удержания

```
In [24]: retention_matrix = cohort_users.pivot(index='cohort_month', columns='lifetime', values='retention_matrix')
```

Out[24]:

lifetime	0	1	2	3	4	5
----------	---	---	---	---	---	---

cohort_month						
2020-09	1.00	0.06	0.02	0.01	0.01	0.00
2020-10	1.00	0.03	0.01	0.01	0.00	NaN
2020-11	1.00	0.02	0.01	0.01	NaN	NaN
2020-12	1.00	0.02	0.01	NaN	NaN	NaN
2021-01	1.00	0.03	NaN	NaN	NaN	NaN
2021-02	1.00	NaN	NaN	NaN	NaN	NaN

```
In [25]: sns.heatmap(retention_matrix, annot=True, fmt='.1%', cmap='Blues')

plt.title('Удержание пользователей по когортам')
plt.xlabel('Время жизни пользователя')
plt.ylabel('Когорта')
plt.show()
```



Вывод

С 0 по 1 месяц теряется от 93.7% до 97.9% процентов пользователей. В последующие месяца отток ослабевает и удержание стабилизируется на уровне от 0.3 % до 1.7 %

Доля пользователей с повторной покупкой

Распределение количества покупок

```
In [26]: purchases = events[events.event_type == 'purchase']

purchases_per_user = (purchases.groupby('user_id')
```



```
.size().reset_index(name='purchases'))  
purchases_per_user['purchases'].describe()
```

```
Out[26]: count    21,304.00  
mean         1.75  
std          1.74  
min          1.00  
25%          1.00  
50%          1.00  
75%          2.00  
max          56.00  
Name: purchases, dtype: float64
```

Доля пользователей совершивших более 1 покупки

```
In [27]: repeat_purchase_rate = ((purchases_per_user['purchases'] > 1).mean())  
repeat_purchase_rate
```

```
Out[27]: np.float64(0.36171610965076983)
```

Распределение количества пользователей по числу покупок

```
In [28]: users_per_purch = purchases.groupby('user_id').size().value_counts()  
users_per_purch
```

```
Out[28]: 1      13598  
2       4292  
3       1691  
4        797  
5        344  
6        209  
7         95  
8         92  
9         47  
10        43  
11         17  
12         14  
13         10  
14          9  
15          7  
18          6  
24          5  
16          4  
17          4  
21          3  
19          3  
20          3  
39          1  
43          1  
33          1  
28          1  
56          1  
36          1  
45          1  
49          1  
22          1  
23          1  
42          1  
Name: count, dtype: int64
```

```
In [29]: users_per_purch.plot(kind='bar', rot=0, figsize=(10,5))  
  
plt.title('Распределение количества пользователей по числу покупок')  
plt.xlabel('Число покупок')  
plt.ylabel('Количество пользователей')
```

```
plt.grid(axis='y', alpha=0.5)  
plt.show()
```



Вывод

Большинство покупателей совершает только одну покупку: 63.8% пользователей не возвращаются за повторной покупкой. При этом 36.2% пользователей совершают более одной покупки. Среднее количество покупок на покупателя составляет 1.75, а медианное значение 1 покупка.

Таким образом, для магазина актуальной задачей является увеличение доли повторных покупок и удержание уже привлеченных покупателей.

Анализ выручки

Расчет месячной выручки

```
In [30]: revenue = purchases['price'].sum()  
revenue
```

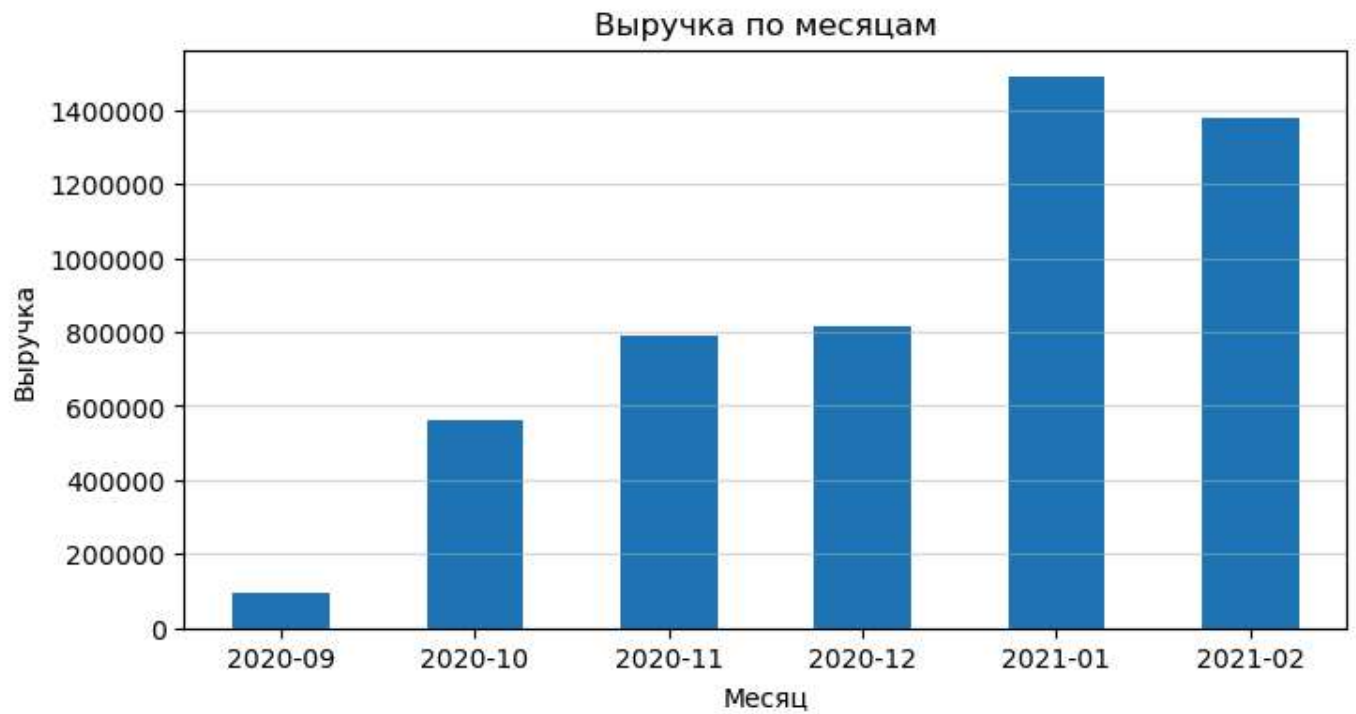
```
Out[30]: np.float64(5125113.92)
```

```
In [31]: revenue_by_month = purchases.groupby('event_month')['price'].sum()  
revenue_by_month
```

```
Out[31]: event_month  
2020-09      96,353.01  
2020-10     562,590.39  
2020-11     787,886.24  
2020-12     813,068.01  
2021-01    1,488,410.57  
2021-02    1,376,805.70  
Freq: M, Name: price, dtype: float64
```

```
In [32]: revenue_by_month.plot(kind='bar', rot=0, figsize=(8,4))  
  
plt.ticklabel_format(style='plain', axis='y')  
  
plt.title('Выручка по месяцам')  
plt.xlabel('Месяц')
```

```
plt.ylabel('Выручка')
plt.grid(axis='y', alpha=0.5)
plt.show()
```



Выручка по когортам

```
In [33]: cohort_revenue = (purchases.groupby(['cohort_month', 'lifetime'])['price'].sum().reset_index(
cohort_revenue
```

Out[33]:

	cohort_month	lifetime	price
0	2020-09	0	96,353.01
1	2020-09	1	26,967.41
2	2020-09	2	8,258.74
3	2020-09	3	580.84
4	2020-09	4	1,383.55
5	2020-09	5	1,521.24
6	2020-10	0	535,622.98
7	2020-10	1	46,835.55
8	2020-10	2	14,722.78
9	2020-10	3	6,874.40
10	2020-10	4	6,991.28
11	2020-11	0	732,791.95
12	2020-11	1	32,087.43
13	2020-11	2	21,581.36
14	2020-11	3	8,753.12
15	2020-12	0	765,676.96
16	2020-12	1	60,195.59
17	2020-12	2	24,004.77
18	2021-01	0	1,398,375.67
19	2021-01	1	113,872.32
20	2021-02	0	1,221,662.97

In [34]:

```
revenue_matrix = cohort_revenue.pivot(index='cohort_month', columns='lifetime', values='price')
revenue_matrix
```

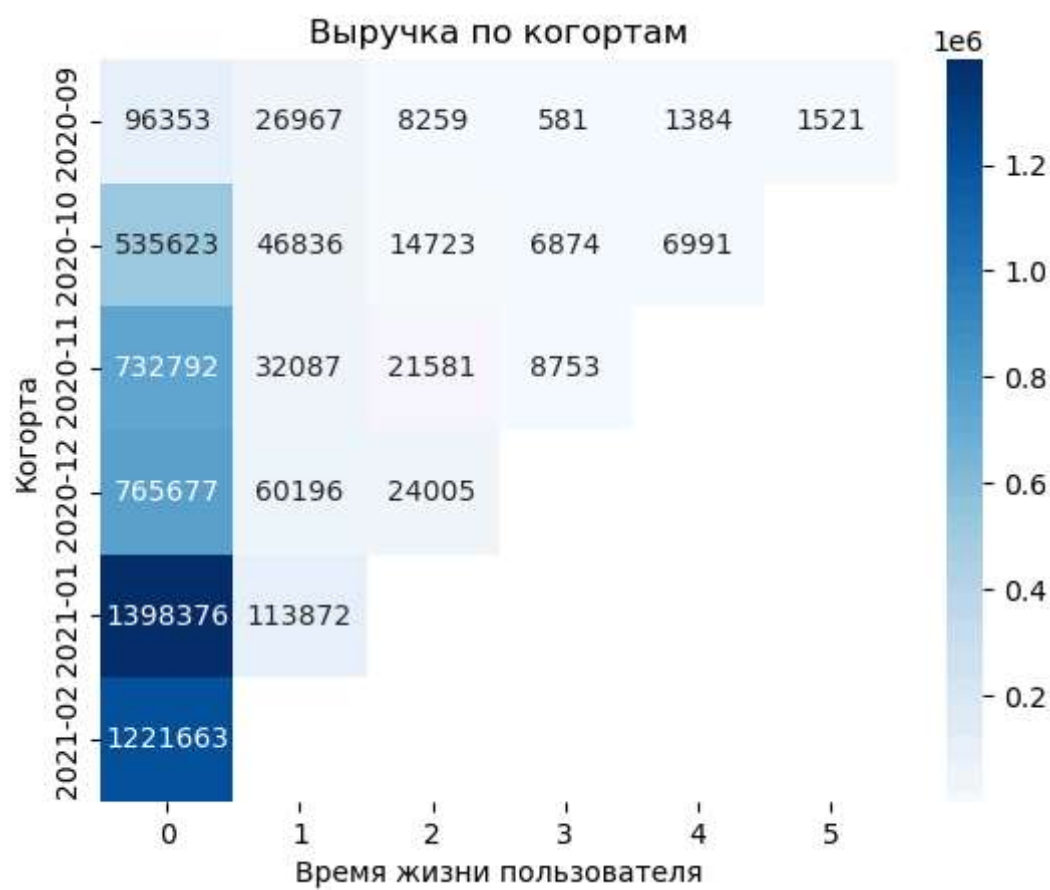
Out[34]:

	lifetime	0	1	2	3	4	5
cohort_month							
2020-09		96,353.01	26,967.41	8,258.74	580.84	1,383.55	1,521.24
2020-10		535,622.98	46,835.55	14,722.78	6,874.40	6,991.28	NaN
2020-11		732,791.95	32,087.43	21,581.36	8,753.12	NaN	NaN
2020-12		765,676.96	60,195.59	24,004.77	NaN	NaN	NaN
2021-01		1,398,375.67	113,872.32	NaN	NaN	NaN	NaN
2021-02		1,221,662.97	NaN	NaN	NaN	NaN	NaN

In [35]:

```
sns.heatmap(revenue_matrix, annot=True, fmt='.0f', cmap='Blues')

plt.title('Выручка по когортам')
plt.xlabel('Время жизни пользователя')
plt.ylabel('Когорта')
plt.show()
```



ARPU по когортам

```
In [36]: cohort_arpu = cohort_revenue.merge(cohort_size.reset_index(name='users_count'), on='cohort_mo
cohort_arpu['ARPU'] = cohort_arpu['price'] / cohort_arpu['users_count']
cohort_arpu
```

Out[36]:

	cohort_month	lifetime	price	users_count	ARPU
0	2020-09	0	96,353.01	15333	6.28
1	2020-09	1	26,967.41	15333	1.76
2	2020-09	2	8,258.74	15333	0.54
3	2020-09	3	580.84	15333	0.04
4	2020-09	4	1,383.55	15333	0.09
5	2020-09	5	1,521.24	15333	0.10
6	2020-10	0	535,622.98	83249	6.43
7	2020-10	1	46,835.55	83249	0.56
8	2020-10	2	14,722.78	83249	0.18
9	2020-10	3	6,874.40	83249	0.08
10	2020-10	4	6,991.28	83249	0.08
11	2020-11	0	732,791.95	90089	8.13
12	2020-11	1	32,087.43	90089	0.36
13	2020-11	2	21,581.36	90089	0.24
14	2020-11	3	8,753.12	90089	0.10
15	2020-12	0	765,676.96	69467	11.02
16	2020-12	1	60,195.59	69467	0.87
17	2020-12	2	24,004.77	69467	0.35
18	2021-01	0	1,398,375.67	78270	17.87
19	2021-01	1	113,872.32	78270	1.45
20	2021-02	0	1,221,662.97	70829	17.25

In [37]:

```
arpu_matrix = cohort_arpu.pivot(index='cohort_month', columns='lifetime', values='ARPU')
arpu_matrix
```

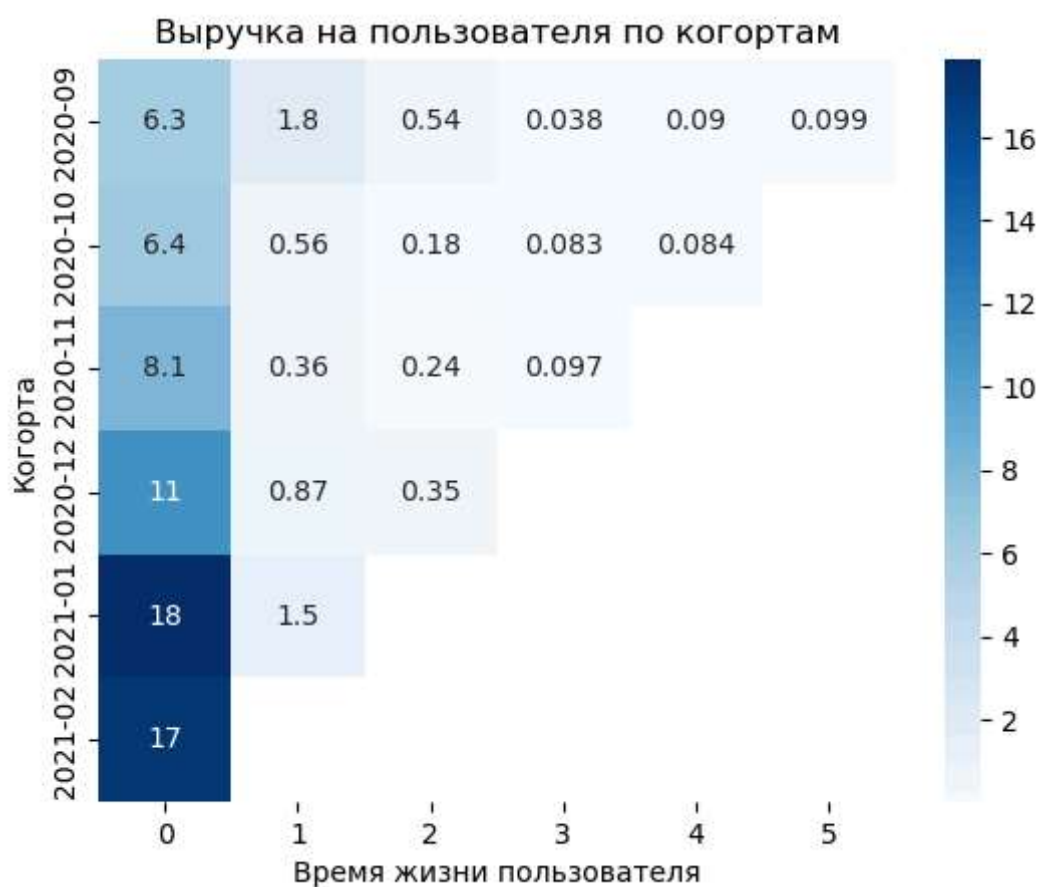
Out[37]:

	lifetime	0	1	2	3	4	5
cohort_month							
2020-09	6.28	1.76	0.54	0.04	0.09	0.10	
2020-10	6.43	0.56	0.18	0.08	0.08	NaN	
2020-11	8.13	0.36	0.24	0.10	NaN	NaN	
2020-12	11.02	0.87	0.35	NaN	NaN	NaN	
2021-01	17.87	1.45	NaN	NaN	NaN	NaN	
2021-02	17.25	NaN	NaN	NaN	NaN	NaN	

In [38]:

```
sns.heatmap(arpu_matrix, annot=True, cmap='Blues')

plt.title('Выручка на пользователя по когортам')
plt.xlabel('Время жизни пользователя')
plt.ylabel('Когорта')
plt.show()
```



Вывод

Для всех когорт концентрация основной части выручки находится в первом месяце и после него сильно снижается. Таким образом, магазин получает основную часть дохода вскоре после привлечения пользователя, а дальнейшие покупки происходят гораздо реже.

Анализ товаров

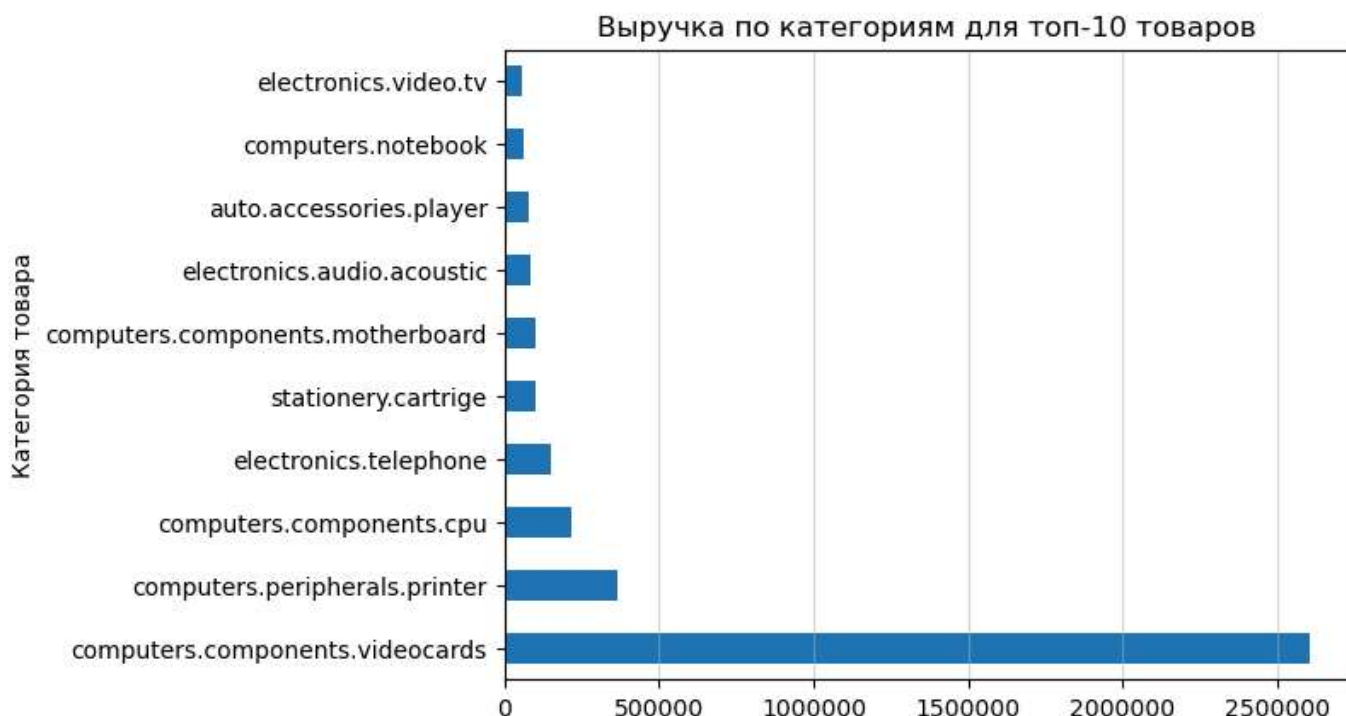
```
In [39]: revenue_by_category = purchases.groupby('category_code')['price'].sum().sort_values(ascending=True)
revenue_by_category.head(10)
```

```
Out[39]: category_code
computers.components.videocards    2,604,764.80
computers.peripherals.printer      364,566.80
computers.components.cpu           219,168.98
electronics.telephone              150,915.69
stationery.cartridge               103,595.99
computers.components.motherboard   102,871.16
electronics.audio.acoustic         84,724.80
auto.accessories.player            81,558.66
computers.notebook                 63,604.79
electronics.video.tv               57,987.91
Name: price, dtype: float64
```

```
In [40]: revenue_by_category.head(10).plot(kind='barh')

plt.ticklabel_format(style='plain', axis='x')

plt.title('Выручка по категориям для топ-10 товаров')
plt.xlabel('')
plt.ylabel('Категория товара')
plt.grid(axis='x', alpha=0.5)
plt.show()
```



Вывод

Основную часть выручки формируют товары категории `computers.components.videocards` — около 2.60 млн - это значительно выше выручки по остальным категориям. На втором и третьем местах находятся `computers.peripherals.printer` и `computers.components.cpu` с выручкой 364.6 тыс. и 219.2 тыс. .

Выручка магазина сильно сконцентрирована на компьютерных комплектующих. Это делает данную категорию ключевым источником дохода, но одновременно повышает зависимость общей выручки от ее продаж.

Итоговый вывод

В ходе исследования проанализировано поведение пользователей интернет-магазина электроники: построена продуктовая воронка, проведен когортный анализ, рассчитан retention, исследованы повторные покупки, выручка и ARPU по когортам, а также структура выручки по категориям.

Основная проблема воронки находится на этапе перехода от просмотра товара к добавлению в корзину: конверсия составляет `9.08%` . При этом из пользователей, добавивших товар в корзину, `57.66%` совершают покупку. Это означает, что наибольший потенциал для роста конверсии находится на этапе `view → cart` .

Когортный анализ показывает высокий отток пользователей после первого периода активности.

Также проблемой является повторная покупка, `63.8%` покупателей совершили только одну покупку. Доля пользователей совершивших более одной покупки составляет - `36.2%` .

ARPU в M0 вырос с `6.28` у сентябрьской когорты до `17.87` у январской, при этом, после первого месяца значение значительно снижается. Месячная выручка также в целом росла и достигла максимального значения в январе 2021 года, после чего в феврале снизилась.

Основным источником дохода является категория `computers.components.videocards` с выручкой около 2.60 млн, значительно опережающая остальные категории. Это делает категорию ключевым источником дохода, но одновременно увеличивает зависимость бизнеса от ее продаж.

Таким образом, основными направлениями для роста являются повышение конверсии из просмотра товара в добавление в корзину view → cart, увеличение доли повторных покупок и работа над удержанием пользователей после первого периода активности. Дополнительно стоит изучить факторы, которые обеспечили более высокий ARPU у поздних когорт/

In []: